

Spring AI: Module 06

RAG Part 2: Advanced Retrieval, Reranking & Evaluation Metrics

Production-Grade Enterprise Retrieval Architectures

1. Why Naive RAG Fails in Production

- Semantic vector search is fuzzy:
 - Can return passages with high semantic similarity but wrong factual specifics (e.g. wrong jurisdiction, obsolete year).
 - Struggles with keyword-exact queries (part numbers, contract IDs).
- **Solution:** Hybrid search + Metadata Filtering + Contextual Reranking.

2. Metadata Filtering in Spring AI

```
SearchRequest request = SearchRequest.builder()  
    .query("termination notice")  
    .topK(10)  
    .filterExpression("jurisdiction == 'NY' AND effectiveYear >= 2024")  
    .build();  
  
List<Document> candidateChunks = vectorStore.similaritySearch(request);
```

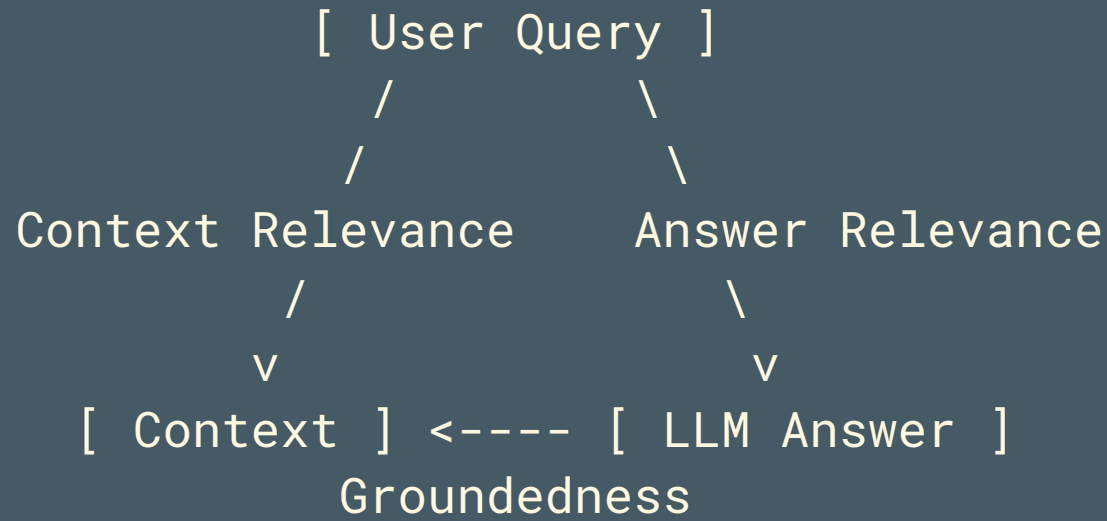
- Filter expressions prevent cross-tenant data leaks and enforce regulatory jurisdiction isolation.

3. Two-Stage Retrieval & Reranking

```
[Query] -> Stage 1: Vector Search (Top-20 broad candidates)  
        -> Stage 2: Cross-Encoder Reranker (Top-3 high-precision chunks)  
        -> Stage 3: LLM Generation
```

- Dramatically boosts Precision@K without the computational overhead of running heavy models across millions of vectors.

4. The RAG Triad of Automated Evaluation



1. **Context Relevance:** Did we fetch what was asked?
2. **Groundedness / Faithfulness:** Is the answer 100% backed by context?
3. **Answer Relevance:** Did the answer directly solve the user prompt?

5. Summary & Best Practices

1. Always filter by metadata first to prune search space.
2. Rerank top candidates before sending to the LLM.
3. Automatically score your RAG pipeline using the Triad to detect hallucinations continuously.